CentraleSupélec

# Rapport de Mission de Fin d'Études

## Turbo décodage par réseaux de neurones

*Neural Network-based Turbo Decoding*


**Henri Bellamy**

| | |
|---|---|
| **École :** | CentraleSupélec |
| **Entreprise :** | Safran Data Systems |
| **Lieu :** | Les Ulis, France |
| **Tuteur académique :** | Simon Leglaive |
| **Maître d'apprentissage :** | Amine Jhaidri |
| **Date :** | 27 Août 2026 |


SAFRAN

Document confidentiel | CentraleSupélec | Safran Data Systems

## Remerciements

Je tiens, à travers ces quelques lignes, à exprimer ma sincère reconnaissance à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de cette mission de fin d'études.

Mes remerciements s'adressent en premier lieu à Amine Jhaidri, mon maître d'apprentissage, pour la confiance qu'il m'a accordée tout au long de ces trois années. Sa disponibilité, sa pédagogie et son expertise en traitement du signal ont été déterminantes dans ma progression. Nos échanges réguliers, toujours riches et constructifs, m'ont permis d'aborder ce sujet exigeant avec méthode et de développer une véritable autonomie de recherche. Je lui suis particulièrement reconnaissant de m'avoir laissé explorer librement mes idées tout en me guidant lorsque cela était nécessaire.

Je souhaite ensuite adresser mes remerciements les plus sincères à Simon Leglaive, mon tuteur académique à CentraleSupélec. Son accompagnement tout au long de cette alternance, ses conseils avisés et son regard extérieur sur mon travail ont été d'une aide précieuse. Sa disponibilité et l'intérêt qu'il a porté à ma mission ont grandement contribué à la qualité de ce projet. Je retiens tout particulièrement la justesse de ses remarques, qui m'ont appris à prendre du recul sur mes résultats et à formuler mes idées avec davantage de clarté. Au-delà des compétences techniques, c'est une véritable démarche scientifique qu'il m'a aidé à construire.

Je remercie également l'ensemble de Safran Data Systems, et tout particulièrement mon équipe, pour son accueil chaleureux et sa bienveillance tout au long de ces trois années. Les échanges quotidiens, qu'ils soient techniques ou informels, ainsi que la bonne humeur des collaborateurs, stagiaires et alternants que j'ai côtoyés, ont rendu cette expérience aussi enrichissante sur le plan professionnel qu'humain.

Enfin, je remercie l'équipe pédagogique de CentraleSupélec, dont la formation m'a donné les bases scientifiques et méthodologiques nécessaires pour mener à bien ce projet.

## Résumé — Abstract

**Résumé**

Les codes correcteurs d'erreurs sont au cœur des systèmes de communication modernes, garantissant une transmission fiable sur des canaux bruités. Les codes convolutifs, et en particulier les turbo codes, ont permis de s'approcher de la limite théorique de Shannon et sont aujourd'hui largement déployés dans les communications satellite, la 5G et les systèmes embarqués.

Au sein de Safran Data Systems, l'équipe CRT (Control, Ranging and Telemetry) conçoit des modems satellite dont la correction d'erreurs repose sur le turbo code, aujourd'hui implémenté en Field-Programmable Gate Array (FPGA). L'équipe souhaite faire évoluer cette offre vers une solution entièrement logicielle, plus souple à déployer, moins coûteuse et affranchie de tout matériel dédié. Dans cette optique, une réécriture directe du turbo décodeur en C++ a été réalisée mais a échoué : l'architecture du turbo code se parallélise mal et n'atteint pas, en logiciel, les débits imposés par les contraintes temps réel du modem. L'enjeu de cette mission est donc d'explorer une voie alternative fondée sur le deep learning, permettant d'accélérer significativement le décodage tout en maintenant des performances de correction acceptables en termes de taux d'erreur binaire (BER, Bit Error Rate).

Le turbo décodage repose sur la coopération itérative de deux décodeurs qui échangent des informations de fiabilité sur chaque bit jusqu'à convergence. C'est précisément chaque décodeur, coûteux en calcul, qui constitue le goulot d'étranglement du passage en logiciel. L'approche retenue consiste à remplacer chacun de ces décodeurs par un réseau de neurones, tout en conservant la structure itérative du turbo code. Les données encodées pouvant être interprétées comme des séries temporelles, l'utilisation de réseaux de neurones récurrents constitue un choix naturel pour capturer les dépendances introduites par le code. D'après le théorème d'approximation universelle, un réseau suffisamment grand peut approximer toute fonction continue avec une précision arbitraire ; il a donc théoriquement la capacité de reproduire le comportement d'un décodeur optimal. De plus, une fois entraîné, un réseau effectue sa prédiction par de simples opérations matricielles, naturellement parallélisables et dont la complexité ne dépend pas de la longueur de contrainte du code, contrairement aux algorithmes de décodage classiques.

Plusieurs approches proposées dans la littérature ont été implémentées et testées, sans atteindre des performances satisfaisantes. Cela nous a conduits à développer notre propre méthode. Le travail s'est structuré en deux étapes. J'ai d'abord conçu DeepMAP, un réseau de neurones reproduisant le comportement d'un unique décodeur convolutif. Une fois cette brique validée, j'ai assemblé deux modules DeepMAP en parallèle, connectés selon le schéma itératif du turbo décodage classique, formant ainsi DeepTurbo, qui reconstitue la chaîne de décodage complète.

Les résultats obtenus valident cette approche. Sur canal à bruit blanc gaussien additif (AWGN, Additive White Gaussian Noise), DeepMAP égale les performances en BER d'un décodeur convolutif classique, et DeepTurbo celles du turbo décodeur de référence. Cette équivalence en performances s'accompagne d'un gain significatif sur le temps de décodage. Ce résultat démontre la viabilité de l'approche par apprentissage profond pour le turbo décodage et lève le principal verrou technique qui bloquait la transition vers une offre logicielle. Pour l'équipe CRT, il constitue une validation concrète de la démarche et une base technique directement réutilisable.

**Abstract**

Error-correcting codes lie at the heart of modern communication systems, ensuring reliable transmission over noisy channels. Convolutional codes, and turbo codes in particular, have brought performance close to the theoretical Shannon limit and are now widely deployed in satellite communications, 5G networks and embedded systems.

Within Safran Data Systems, the CRT (Control Ranging and Telemetry) team designs satellite modems whose error correction relies on a turbo code, currently implemented in Field-Programmable Gate Array (FPGA). The team aims to transition towards a fully software-based solution, easier to deploy, less costly and free from dedicated hardware. A straightforward rewrite of the turbo decoder in C++ was attempted but proved unsuccessful: the turbo code architecture is inherently difficult to parallelise and, in software, fails to meet the real-time throughput requirements of the modem. The purpose of this project is therefore to explore an alternative approach based on deep learning, capable of significantly accelerating the decoding process while maintaining acceptable error-correction performance in terms of Bit Error Rate (BER).

Turbo decoding relies on the iterative cooperation of two convolutional decoders that exchange reliability information on each bit until convergence. Each convolutional decoder is computationally expensive and constitutes the bottleneck for the transition to software. The chosen approach consists in replacing each of these decoders with a neural network, while preserving the iterative structure of the turbo code. Since encoded data can be interpreted as time series, recurrent neural networks are a natural choice for capturing the dependencies introduced by the code. According to the universal approximation theorem, a sufficiently large network can approximate any continuous function with arbitrary precision; it therefore has, in theory, the capacity to reproduce the behaviour of an optimal decoder. Moreover, once trained, a neural network performs its prediction through simple matrix operations that are inherently parallelisable and whose complexity does not depend on the constraint length of the code, unlike conventional decoding algorithms.

Several approaches proposed in the literature were implemented and tested, without achieving satisfactory performance. This led us to develop our own method. The work was carried out in two stages. I first designed DeepMAP, a neural network that reproduces the behaviour of a single convolutional decoder. Once this building block was validated, two DeepMAP modules were assembled in parallel, connected according to the iterative turbo decoding scheme, thus forming DeepTurbo, which reconstructs the complete decoding chain.

The results validate this approach. Over an Additive White Gaussian Noise (AWGN) channel, DeepMAP matches the BER performance of a conventional convolutional decoder, and DeepTurbo that of the reference turbo decoder. This performance equivalence comes with a significant speedup in decoding time. This result demonstrates the viability of deep learning for turbo decoding and removes the main technical barrier that was preventing the transition to a software-based product. For the CRT team, it provides both a concrete validation of the approach and a directly reusable technical foundation.

## Table des matières

**1 Introduction et reformulation du problème** **7**
1.1 Contexte . . . 7
1.2 Objectifs . . . 7
1.3 Enjeux . . . 7
1.4 Reformulation du problème . . . 8

**2 Approche et analyse** **8**
2.1 État de l'art du turbo code . . . 8
2.1.1 Chaine de communication . . . 8
2.1.2 Turbo encodeur . . . 9
2.1.3 Turbo décodeur . . . 9
2.2 État de l'art du décodage par deep learning . . . 10
2.2.1 Approche par initialisation BCJR et fine-tuning . . . 10
2.2.2 Approche par DeepSISO spécialisés par itération . . . 10
2.2.3 Approche bout en bout sans initialisation . . . 10
2.3 Positionnement de l'approche retenue . . . 11
2.3.1 Architecture de DeepMAP . . . 11
2.3.2 Méthode d'entraînement . . . 12
2.4 Organisation du travail . . . 13
2.5 Résultats intermédiaires . . . 14
2.6 Difficultés rencontrées et solutions apportées . . . 16
2.6.1 Reproductibilité des résultats de la littérature. . . . 17
2.7 Gestion des risques . . . 17

**3 Résultats** **17**
3.1 Validité des résultats . . . 17
3.1.1 Performance de décodage de DeepTurbo . . . 17
3.1.2 Vitesse de décodage de DeepTurbo . . . 19
3.2 Caractère innovant . . . 20
3.3 Utilité et réutilisabilité des résultats . . . 20
3.4 Limites et suites à donner . . . 20
3.5 Quantification de la valeur ajoutée . . . 21

**4 Prolongements possibles** **21**
4.1 TurboNet : un réseau unique non itératif . . . 21
4.2 Turbo Auto-Encodeur . . . 21
4.3 Généralisation à d'autres modèles de canal . . . 21

**5 Bilan des apprentissages professionnels** **22**
5.1 Compétences techniques acquises . . . 22
5.2 Compétences transversales . . . 22
5.3 Mise en perspective pour le futur professionnel . . . 23

**6 Prise de recul sur les enjeux éthiques** **23**
6.1 Enjeux individuels et responsabilité de l'ingénieur . . . 23
6.2 Enjeux collectifs et sociétaux . . . 23
6.3 Développement durable et RSE . . . 23
6.4 Diversité et interculturalité . . . 24

**A Rappels sur les turbo codes** **26**
A.1 Encodeur convolutif RSC (7, 5) . . . 26
A.2 Structure de l'entrelaceur . . . 26
A.3 Algorithme BCJR . . . 26

A.4 Information extrinsèque . . . . . 27

**B Protocole d'évaluation BER** **27**

**C Glossaire** **27**

## Introduction et reformulation du problème

### Contexte

L'équipe Control, Ranging and Telemetry (CRT) de Safran Data Systems est spécialisée dans la conception de modems pour satellites, actuellement implémentés en technologie Field Programmable Gate Array (FPGA). En son sein, je fais partie de la branche Traitement du Signal (TDS), dirigée par Amine Jhaidri, ingénieur expert en traitement du signal et maître d'apprentissage. Cette équipe de six personnes mène les activités R&D sur les algorithmes de traitement du signal et de décodage. L'équipe s'inscrit aujourd'hui dans une transition stratégique du CRT : faire évoluer les modems vers une offre entièrement logicielle afin de réduire les coûts, de simplifier le déploiement et de répondre à une demande croissante du marché pour des solutions sans matériel dédié.

La chaîne de communication du modem utilise un turbo code [1] pour corriger les erreurs de transmission. Le passage en logiciel de ce turbo décodeur a déjà été exploré en C++, mais les résultats se sont révélés insuffisants : le décodage turbo est par nature itératif et séquentiel, ce qui le rend difficile à paralléliser et trop lent pour répondre aux contraintes temps réel du modem. C'est dans ce contexte que s'inscrit la présente mission : étudier ce que le deep learning peut apporter pour accélérer le décodage.

### Objectifs

**Objectif principal**

Concevoir, entraîner et évaluer un réseau de neurones capable de reproduire le comportement du turbo décodeur, en visant des performances BER comparables à celles du décodeur classique sur canal AWGN, tout en offrant un gain significatif en vitesse de décodage.

Les objectifs secondaires sont :

- Établir un état de l'art du décodage par apprentissage profond [2], [3], [4]
- Quantifier le compromis BER / temps d'inférence / taille du modèle
- Produire un prototype et une base de résultats réutilisables par l'équipe TDS

### Enjeux

La mission porte des enjeux à plusieurs niveaux.

Sur le plan technique, la question fondamentale est d'abord celle de la faisabilité : un réseau de neurones est-il seulement capable de reproduire le comportement du turbo décodeur ? Au-delà, il s'agit de caractériser le compromis entre performances de correction et vitesse de décodage, l'objectif étant d'obtenir le plus haut débit possible tout en maintenant un BER acceptable.

Sur le plan stratégique, la réussite de cette exploration validerait l'approche par apprentissage profond comme voie crédible pour la transition du modem FPGA vers une offre logicielle. Elle fournirait à l'équipe CRT un outil technique concret pour poursuivre cette transition et répondre à la demande client d'une solution déployable sans châssis matériel dédié.

Sur le plan scientifique, le décodage de turbo codes par réseaux de neurones reste un sujet de recherche ouvert, et la mission offre l'occasion de confronter les résultats de la littérature à un cas d'usage industriel réel.

### Reformulation du problème

**Problématique**

Un réseau de neurones peut-il reproduire les performances du turbo décodeur (en termes de BER) tout en offrant une architecture suffisamment parallélisable pour permettre un gain significatif en vitesse de décodage, ouvrant ainsi la voie à un déploiement logiciel ?

Cette reformulation fait apparaître deux axes principaux. Le premier est celui de la performance : le réseau doit atteindre un BER comparable à celui du décodeur classique. Le second est celui de la vitesse : l'architecture doit tirer parti du traitement par batchs et du décodage par produits matriciels pour offrir un débit de décodage compatible avec les exigences du modem. Le meilleur compromis entre ces deux axes constitue le cœur de la mission.

## Approche et analyse

### État de l'art du turbo code

*Chaine de communication*

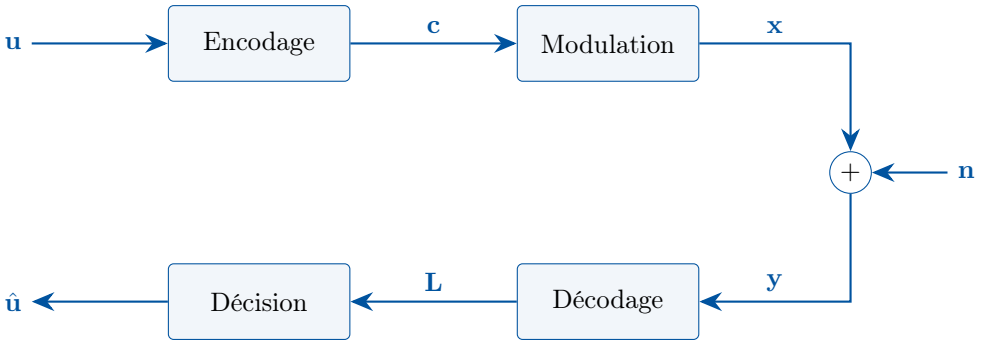



FIGURE 1 – Chaîne de communication

La figure 1 illustre la chaîne de communication considérée. Un message de $k$ bits $\mathbf{u} \in \{0,1\}^k$ est encodé en un mot de code de $N$ bits $\mathbf{c} \in \{0,1\}^N$. Les bits encodés sont ensuite modulés en Binary Phase Shift Keying (BPSK) pour produire le signal transmis $\mathbf{x} \in \{-1,+1\}^N$. Lors de la transmission, un bruit AWGN est ajouté au signal, de sorte que le signal reçu s'écrit :

$$\mathbf{y} = \mathbf{x} + \mathbf{n}$$

où $\mathbf{n} \sim \mathcal{N}(0,\sigma^2)$ représente un bruit gaussien de moyenne nulle et de variance $\sigma^2$. Le canal AWGN est le modèle de référence pour évaluer les performances des systèmes de communication en conditions de bruit standard. Le décodeur traite le signal reçu $\mathbf{y}$ et produit les rapports de vraisemblance logarithmiques (LLR, Log-Likelihood Ratio) $\mathbf{L} \in \mathbb{R}^k$, à partir desquels une décision est appliquée pour obtenir l'estimation $\hat{\mathbf{u}}$ du message original selon la règle :

$$\hat{u}_i = \begin{cases} 1 & \text{si } L_i > 0 \\ 0 & \text{sinon} \end{cases}$$

Les performances se mesurent en comparant $\mathbf{u}$ et $\hat{\mathbf{u}}$ par le BER en fonction du rapport signal sur bruit (SNR, Signal-to-Noise Ratio). Le SNR, exprimé en décibels, est défini par :

$$\mathrm{SNR}_{\mathrm{dB}} = 10\log_{10}\frac{E_b}{N_0}$$

où $E_b$ est l'énergie par bit et $N_0$ la densité spectrale de puissance du bruit. Plus le SNR est élevé, moins le canal est bruité, et plus le décodage est aisé.

*Turbo encodeur*

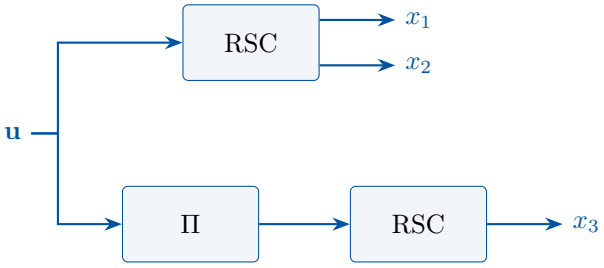



FIGURE 2 – Architecture du turbo encodeur

La figure 2 présente le turbo encodeur. Il est composé de deux encodeurs convolutifs récursifs systématiques (RSC) de rendement 1/2 et d'un entrelaceur Π. Le message **u** est transmis directement au premier encodeur RSC, qui produit le bit systématique $x_1 = u_k$ ainsi qu'un bit de parité $x_2$. Simultanément, **u** est permuté par l'entrelaceur Π, qui réordonne les bits selon un ordre déterministe afin de décorréler les motifs d'erreurs entre les deux encodeurs. La version permutée est ensuite transmise au second encodeur RSC, identique au premier, qui produit un second bit de parité $x_3$. La trame transmise est donc constituée du triplet $(x_1, x_2, x_3)$, d'où le rendement global de 1/3. Dans notre cas, les polynômes générateurs sont $f_1(x) = 1 + x + x^2$ et $f_2(x) = 1 + x^2$, correspondant à la configuration $(7, 5)$ en notation octale.

*Turbo décodeur*

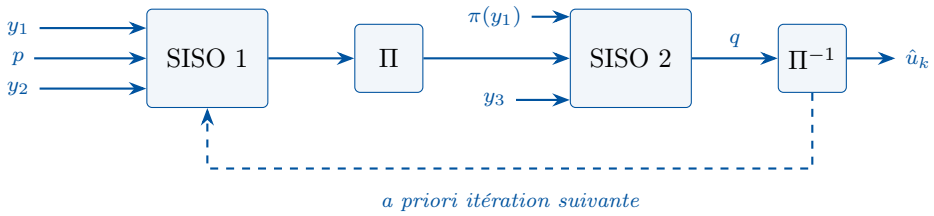



FIGURE 3 – Une itération du turbo décodeur. Le processus est répété $N$ fois jusqu'à convergence.

La figure 3 présente le turbo décodeur [5]. Son fonctionnement repose sur le principe turbo : deux décodeurs à entrée et sortie souples (SISO, Soft-In Soft-Output) raffinent itérativement l'information a posteriori en s'échangeant des estimations de fiabilité à travers l'entrelaceur. Chaque décodeur SISO implémente l'algorithme BCJR [6], qui opère selon un critère de maximum a posteriori (MAP) : il reçoit le signal bruité ainsi qu'une information a priori $p$, et produit une information a posteriori $q$ qui devient l'a priori du décodeur suivant.

Le décodage procède ainsi. À la première itération, l'a priori est nul. Le décodeur 1 reçoit les observations bruitées $(y_1, y_2)$ et l'a priori, puis produit une information a posteriori. Après entrelacement par Π, celle-ci devient l'a priori du décodeur 2, qui reçoit également la version entrelacée du signal systématique $\pi(y_1)$ et la parité $y_3$. Le décodeur 2 produit à son tour une information a posteriori qui, après désentrelacement par $\Pi^{-1}$, devient l'a priori du décodeur 1 pour l'itération suivante. À chaque itération, les estimations gagnent en précision. Après une dizaine d'itérations, une décision sur le signe de l'information a posteriori finale fournit l'estimée $\hat{u}_k$.

L'objectif de cette mission est de remplacer le turbo décodeur classique par une approche fondée sur le deep learning. L'idée est qu'un réseau de neurones, une fois entraîné, effectue son décodage par de simples produits matriciels, bien plus rapides à exécuter que le décodage itératif classique. Reste à déterminer comment concevoir et entraîner un tel réseau : plusieurs approches ont été proposées dans la littérature, que nous présentons dans la section suivante.

### État de l'art du décodage par deep learning

L'idée centrale, commune à l'ensemble de la littérature, consiste à entraîner un réseau de neurones capable de reproduire le comportement d'un décodeur SISO, que l'on notera par la suite DeepSISO, puis à assembler deux de ces réseaux en parallèle selon le schéma itératif du turbo décodage. À chaque itération, les réseaux échangent leurs estimations de fiabilité et l'information a posteriori converge progressivement, à l'image du décodeur classique. Trois approches principales se distinguent dans la façon de concevoir et d'entraîner ce réseau DeepSISO.

*Approche par initialisation BCJR et fine-tuning*

La première approche [4] consiste à initialiser les poids du DeepSISO à partir de ceux de l'algorithme BCJR, de sorte que le réseau reproduise fidèlement le décodeur classique dès le départ. Un réseau hybride est ensuite construit, composé d'une partie apprenable (les poids du réseau) et d'une partie figée (l'entrelaceur, la structure itérative, le calcul de l'information échangée entre décodeurs). Ce réseau prend en entrée la trame bruitée et reproduit le schéma turbo complet. Une seconde phase d'entraînement, dite de fine-tuning, réentraîne l'ensemble en minimisant non plus l'erreur sur les LLR mais directement le BER, afin d'affiner les poids au-delà de ce que le BCJR permet.

*Approche par DeepSISO spécialisés par itération*

La deuxième approche [7] conserve la structure hybride mais spécialise chaque DeepSISO pour une itération donnée. Pour un turbo décodage à dix itérations, dix réseaux distincts sont entraînés : chacun reçoit la trame bruitée et l'information a priori propre à son itération, et apprend à prédire l'information a posteriori correspondante. Le dernier DeepSISO est entraîné à prédire directement le bit décodé. Chaque réseau étant optimisé pour les conditions statistiques de son itération, il reproduit au mieux le BCJR dans ce contexte précis. L'ensemble peut ensuite faire l'objet d'un fine-tuning global.

*Approche bout en bout sans initialisation*

La troisième approche [8] s'affranchit de toute initialisation par le BCJR. Les dix DeepSISO sont assemblés en structure turbo et entraînés de bout en bout. Une différence notable réside dans la représentation de l'information échangée : au lieu d'un scalaire par bit, chaque bit est représenté par un vecteur de $\mathbb{R}^d$ (typiquement $d = 5$), de sorte que l'information échangée $\mathbf{L}$ est de dimension $(k, d)$ au lieu de $(k, )$. Cette représentation vectorielle offre une information plus riche par bit. Plusieurs stratégies de communication entre itérations peuvent être envisagées. Notons $q_t$ l'information a posteriori produite à l'itération $t$, $p_t$ l'information a priori reçue, et $L_c y_1$ l'observation systématique du canal.

Extrinsèque classique :

$$p_{t+1} = q_t - p_t - L_c y_1$$

Extrinsèque réduite :

$$p_{t+1} = q_t - p_t$$

Posterior as prior :

$$p_{t+1} = q_t$$

Ces trois approches, issues de la littérature, ont chacune été implémentée, étudiée et testée au cours de la mission. Aucune n'a cependant permis d'atteindre des performances suffisantes pour envisager un remplacement du turbo décodeur classique. Ces résultats nous ont conduits à développer notre propre DeepSISO, doté d'une architecture et d'une méthode d'entraînement spécifiques, que nous avons nommé DeepMAP. Sa conception est détaillée dans les sections suivantes.

### Positionnement de l'approche retenue

*Architecture de DeepMAP*

Le tableau 1 présente l'architecture de DeepMAP. Le réseau reçoit en entrée la séquence bruitée de dimension $(k, 3)$, correspondant aux deux bits d'encodage et à l'information a priori. Une couche Gated Recurrent Unit (GRU) [9] bidirectionnelle de 100 unités capture les dépendances temporelles, produisant en sortie, pour chaque pas de temps, un vecteur de dimension 200 (les 100 cellules dans chacun des deux sens de propagation). Une couche convolutive 1D de 32 filtres avec un noyau de taille 5 extrait ensuite des motifs locaux, capturant les dépendances à court terme. Une couche dense de 32 unités combine ces caractéristiques, puis une dernière couche dense à une seule unité produit, pour chaque pas de temps, une valeur réelle interprétable comme un LLR, à partir duquel un seuillage fournit le bit décodé.

Table 1 – Architecture de DeepMAP

| Couche | Activation | Cellules | Noyau | Sortie |
|---|---|---|---|---|
| Input | – | – | – | $(B, k, 3)$ |
| Bi-GRU | tanh | 100 | – | $(B, k, 200)$ |
| Conv1D | ReLU | 32 | 5 | $(B, k, 32)$ |
| Dense | ReLU | 32 | – | $(B, k, 32)$ |
| Dense | linéaire | 1 | – | $(B, k, 1)$ |

**Couche récurrente.** La propagation bidirectionnelle de l'état caché est essentielle pour atteindre des performances de décodage proches de l'optimal. Ce constat, largement documenté dans la littérature [4] [8], a été confirmé par nos propres expériences. Nous employons une couche GRU bidirectionnelle, qui permet d'atteindre ces performances tout en conservant un coût de calcul raisonnable. À chaque pas de temps $t$, la cellule GRU calcule :

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t] + b_z\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t] + b_r\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t \odot h_{t-1}, x_t] + b\right)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

où $z_t$ est la porte de mise à jour, $r_t$ la porte de réinitialisation, $\tilde{h}_t$ l'état candidat, $\sigma$ la fonction sigmoïde et $\odot$ le produit élément par élément. Ce mécanisme de portes permet au réseau de sélectivement conserver ou oublier l'information passée, ce qui est particulièrement adapté à la capture des dépendances temporelles du code convolutif.

L'objectif est de traiter une séquence d'entrée composée de triplets bruités et de produire en sortie une séquence de vecteurs de $\mathbb{R}^{200}$ : chacune des 100 cellules GRU produit une valeur par pas de temps, et la bidirectionnalité double cette dimension. Cette formulation correspond à un modèle séquence-à-séquence dans lequel chaque élément de la séquence, initialement représenté dans $\mathbb{R}^3$, est projeté dans un espace de dimension supérieure afin d'en extraire une représentation plus riche et de mieux capturer les dépendances temporelles du code.

Plusieurs hyperparamètres ont été explorés pour concevoir une architecture efficace. Le nombre de cellules a été exploré de manière exponentielle de 1 à 256 : en dessous de 50 cellules, les performances sont insuffisantes, tandis qu'au-delà de 128 le gain devient négligeable face à l'augmentation de la complexité. La valeur de 100 cellules a été retenue.

**Couche convolutive.** En sortie de la couche récurrente, une couche convolutive 1D applique 32 filtres de taille 5. Pour chaque filtre $j$, la sortie s'écrit :

$$\mathbf{h}^{(j)} = \varphi\left(\mathbf{w}^{(j)} * \mathbf{z} + b^{(j)}\right)$$

où $\varphi$ est la fonction d'activation, ici $\varphi = \text{ReLU}$. $\mathbf{z} \in \mathbb{R}^{k\times 200}$ est la sortie de la couche GRU, $w_{i,c}^{(j)}$ les poids du filtre et $b^{(j)}$ le biais. Chaque filtre opère sur une fenêtre de 5 pas de temps consécutifs, capturant des dépendances locales complémentaires de celles apprises par la couche récurrente. La séquence est ainsi transformée de $\mathbb{R}^{k\times 200}$ vers $\mathbb{R}^{k\times 32}$.

**Couches denses.** La séquence de $k$ vecteurs de $\mathbb{R}^{32}$ produite par la couche convolutive est traitée par deux couches denses appliquées indépendamment à chaque pas de temps. La première, de 32 unités avec activation ReLU, combine les caractéristiques extraites. La seconde, à une seule unité sans activation, produit pour chaque bit un scalaire interprétable comme un LLR, à partir duquel un seuillage fournit le bit décodé.

**Normalisation et régularisation.** L'architecture retenue n'intègre ni couche de batch normalisation ni dropout. Les données d'entrée étant déjà normalisées (moyenne nulle et variance fixée par le canal AWGN), et le volume de données d'entraînement étant suffisamment important pour limiter le surapprentissage, ces techniques se sont avérées inutiles voire contre-productives lors de nos expériences.

*Méthode d'entraînement*

**Jeu de données.** Le jeu de données d'entraînement est constitué de paires $(\mathcal{Y}, \mathcal{L})$, où $\mathcal{Y}$ contient les trames bruitées reçues et $\mathcal{L}$ les LLR de référence correspondants, calculés par l'algorithme BCJR. Le réseau apprend ainsi à reproduire la sortie du décodeur classique : à partir d'une trame bruitée, il doit prédire les LLR que le BCJR aurait produits. Chaque échantillon d'entrée est de dimension $(1784, 3)$, constitué du bit systématique, d'un bit de parité et de l'information a priori associée, tandis que la cible correspondante est de dimension $(1784, 1)$, contenant un LLR par bit.

**Fonction de perte.** Le problème de décodage est ici formulé comme un problème de régression : le réseau cherche à approximer les LLR a posteriori calculés par l'algorithme BCJR, c'est-à-dire des valeurs continues dans $\mathbb{R}$. La fonction de perte retenue est l'erreur quadratique moyenne (MSE, Mean Squared Error) :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{k}\sum_{i=1}^{k}\left(L_i - \hat{L}_i\right)^2$$

où $L_i$ est le LLR de référence calculé par l'algorithme BCJR et $\hat{L}_i$ la prédiction du réseau pour le $i$-ème bit. Minimiser ce critère revient à entraîner le réseau à reproduire le plus fidèlement possible la fonction de décodage optimale au sens du maximum a posteriori.

**Métrique d'évaluation.** La métrique retenue pour évaluer la qualité de l'approximation est l'erreur absolue moyenne (MAE, Mean Absolute Error) :

$$\text{MAE} = \frac{1}{k}\sum_{i=1}^{k}\left|L_i - \hat{L}_i\right|$$

Une MAE proche de zéro signifie que les LLR prédits par le réseau sont quasiment identiques à ceux du BCJR, et donc que DeepMAP reproduit fidèlement le décodeur classique.

**Hyperparamètres d'entraînement.** Plusieurs optimiseurs ont été testés (SGD, RMSprop, Adam). L'optimiseur Adam a systématiquement offert les meilleures performances de convergence et de décodage, avec un taux d'apprentissage de $10^{-4}$. La taille de batch $B$ a été fixée à 16 séquences et la convergence est atteinte en 30 époques. Concernant le SNR d'entraînement, l'usage standard consiste à échantillonner les données d'entraînement et de test à un même SNR. Cependant, nos expériences ont montré que le réseau atteint ses meilleures performances lorsqu'il est entraîné sur des trames bruitées à bas SNR ; pour notre architecture et notre code, le SNR optimal s'est révélé être de 0.5 dB. La longueur des trames d'entraînement est contrainte par l'entrelaceur du turbo code, qui n'accepte que des trames de 1784 bits. La construction du jeu de données nécessite en effet de calculer l'information a priori via le turbo décodeur classique, ce qui impose de travailler avec des trames complètes. Chaque échantillon d'entraînement est donc de dimension $1784 \times 3$ et correspond à une séquence de triplets, chaque triplet contenant les deux bits encodés et l'information a priori associée. Enfin, la taille du jeu de données influence directement les performances : un jeu de 4800 trames encodées et bruitées offre le meilleur compromis entre généralisation et temps d'entraînement. L'ensemble de ces hyperparamètres est résumé dans le tableau 2.

Table 2 – Hyperparamètres d'entraînement de DeepMAP

| **Hyperparamètre** | **Valeur** |
|---|---|
| Optimiseur | Adam |
| Taux d'apprentissage | $10^{-4}$ |
| Taille de batch | 16 |
| Époques | 30 |
| SNR d'entraînement | 0.5 dB |
| Longueur de trame | 1784 bits |
| Taille du dataset | 4800 trames |
| Fonction de perte | MSE |

### Organisation du travail

L'ensemble du développement a été réalisé sur un MacBook Pro 2023 équipé d'une puce M2 Pro (12 cœurs CPU, 12 cœurs GPU). Le GPU n'étant pas compatible avec TensorFlow, backend utilisé par Keras, tous les entraînements ont été effectués sur CPU. Le travail est organisé sous forme de notebooks Jupyter : des scripts Python génèrent les jeux de données, puis les réseaux sont créés, entraînés et évalués depuis le notebook à l'aide de Keras et NumPy.

Le travail quotidien s'articule autour d'un cycle d'expérimentation récurrent. La matinée débute par l'analyse des résultats des entraînements lancés la veille : évaluation du BER des nouveaux modèles, interprétation des courbes d'apprentissage et identification des axes d'amélioration. La suite de la journée est consacrée à l'implémentation de nouvelles architectures, à l'exploration de méthodes alternatives et à l'évaluation de nouveaux jeux de données. En fin de journée, de nouveaux entraînements sont lancés avec une analyse des résultats attendus, afin d'alimenter le cycle du lendemain.

Des points informels avec Amine Jhaidri, maître d'apprentissage, ont lieu tous les deux à trois jours. Ces échanges portent sur la discussion des derniers résultats, la recherche de pistes d'amélioration et l'exploration de nouvelles méthodes identifiées dans la littérature. Bien que je sois le seul membre de l'équipe TDS à travailler sur des sujets de deep learning, des échanges ponctuels avec d'autres collaborateurs du groupe ayant travaillé sur des problématiques similaires ont également nourri la réflexion.

### Résultats intermédiaires

La première étape de la mission a consisté à valider la faisabilité de l'approche : un réseau de neurones est-il capable d'égaler les performances d'un décodeur BCJR classique ? À ce stade, seule la qualité de décodage importe ; les questions d'architecture optimale et de temps de calcul seront traitées dans un second temps.

**Performances en BER.** La figure 4 compare les courbes de BER de DeepMAP et du décodeur BCJR sur canal AWGN, pour une plage de SNR allant de 0 à 7dB. Les deux courbes se superposent sur l'ensemble de la plage, confirmant que DeepMAP reproduit fidèlement le comportement du décodeur classique.

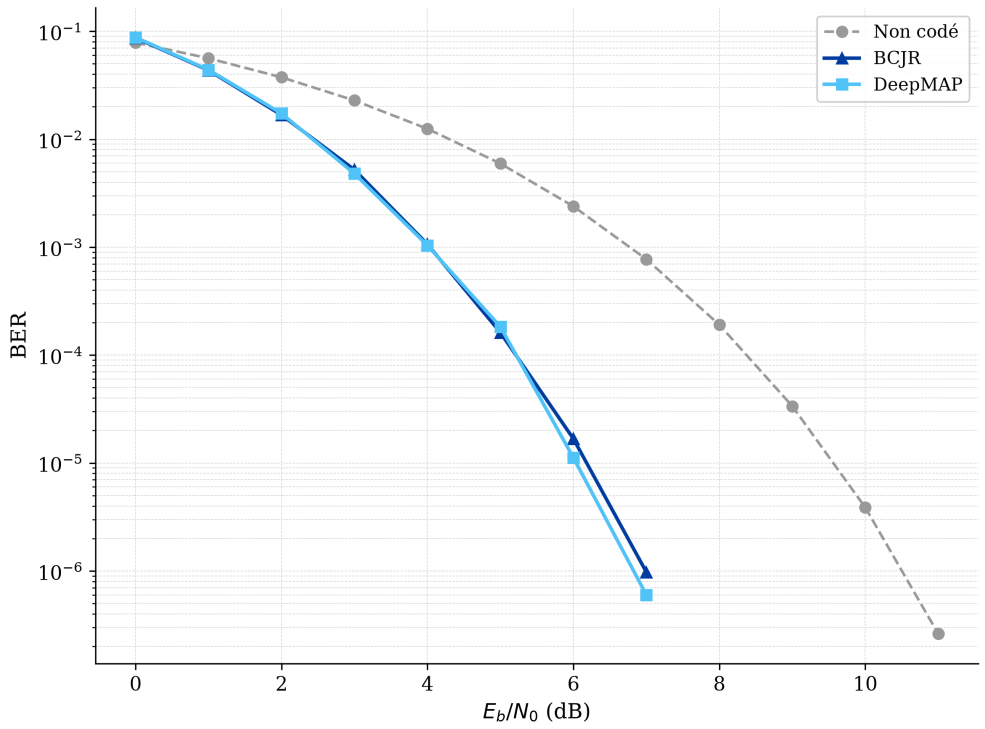



FIGURE 4 – Courbes de BER de DeepMAP et du décodeur BCJR sur canal AWGN

**Qualité de l'approximation.** Le tableau 3 quantifie la fidélité de DeepMAP par rapport au BCJR. La MAE de 0.34 et le coefficient de détermination $R^2$ de 0.97 confirment que les LLR prédits par le réseau sont très proches de ceux calculés par l'algorithme classique.

TABLE 3 – Métriques d'approximation de DeepMAP par rapport au BCJR

| Métrique | Valeur |
|---|---|
| MAE | 0.3397 |
| MSE | 0.1955 |
| $R^2$ | 0.9697 |

**Temps de décodage.** Au-delà de la fidélité, DeepMAP présente un avantage significatif en vitesse. La figure 5 compare les temps de décodage par trame de DeepMAP et du BCJR pour trois longueurs de trame ($10^2$, $10^3$ et $10^4$ bits). Le réseau, une fois entraîné, effectue sa prédiction par de simples produits matriciels [10] [11], ce qui se traduit par un temps de décodage systématiquement inférieur à celui de l'algorithme classique, quelle que soit la taille de la trame. On observe par ailleurs que le BCJR traite chaque bit séquentiellement : son temps de décodage croît linéairement

avec la longueur de la trame. DeepMAP, en revanche, présente une croissance sous-linéaire, rendue possible par la parallélisation des opérations matricielles. Cet écart se creuse donc à mesure que la taille de la trame augmente.

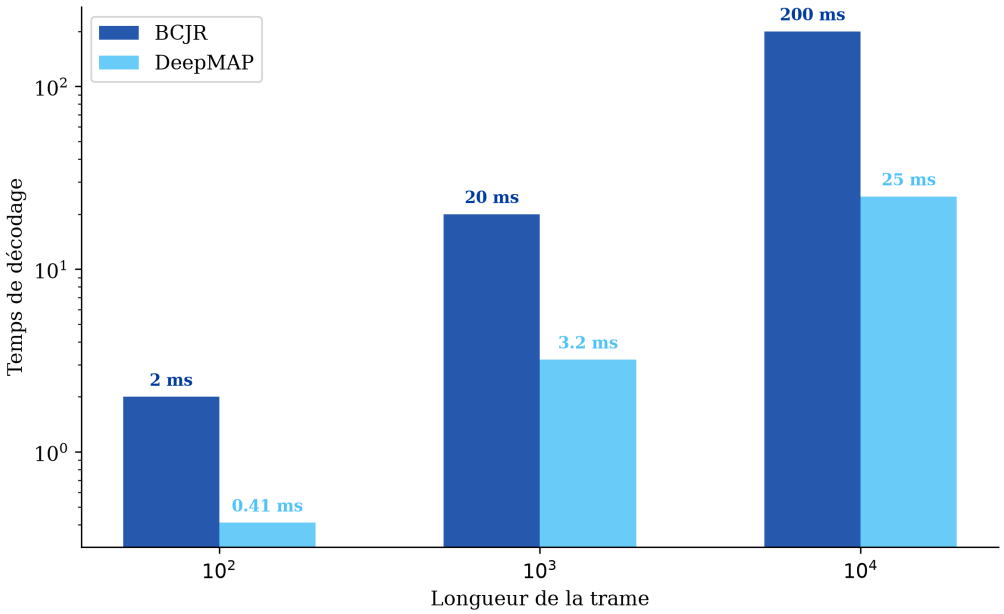



FIGURE 5 – Temps de décodage par trame : DeepMAP vs BCJR

La figure 6 quantifie ce phénomène. Pour une trame de $10^2$ bits, DeepMAP est environ 4.9 fois plus rapide que le BCJR. Ce facteur passe à 6.3 pour une trame de $10^3$ bits et atteint 8.0 pour une trame de $10^4$ bits. Plus la trame est longue, plus l'avantage de DeepMAP se renforce.

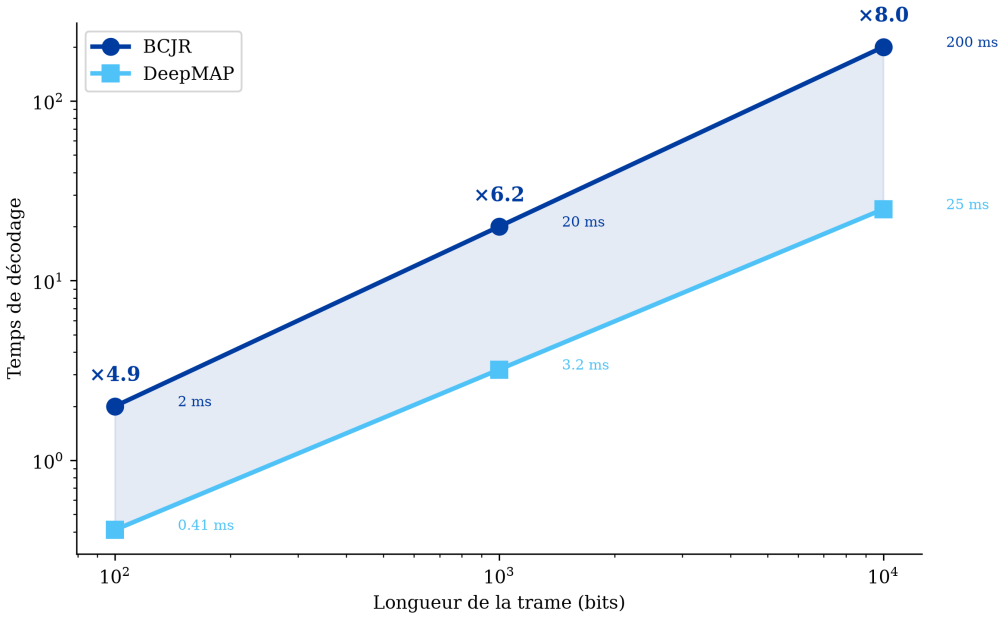



FIGURE 6 – Évolution du facteur d'accélération : DeepMAP vs BCJR

Ces mesures portent sur le décodage d'une seule trame à la fois. En pratique, plusieurs trames arrivent en flux continu : là où le BCJR, séquentiel, doit les traiter l'une après l'autre, DeepMAP peut les regrouper en un batch et les décoder en un seul appel, les produits matriciels traitant tout le batch en parallèle. Le temps par trame diminue alors encore, laissant entrevoir un gain supérieur en conditions réelles d'exploitation.

Ces résultats intermédiaires valident la capacité d'un réseau de neurones à reproduire le décodeur SISO avec une fidélité élevée et un gain en vitesse. L'étape suivante consiste à assembler deux DeepMAP en structure turbo afin de vérifier que ces performances se maintiennent dans le cadre du turbo décodage complet.

### Difficultés rencontrées et solutions apportées

La conception de DeepMAP fait intervenir deux niveaux d'optimisation. Le premier, interne, consiste à trouver les poids $\mathbf{w}^*$ du réseau minimisant la fonction de perte pour un jeu d'hyperparamètres fixé $\boldsymbol{\theta}$ :

$$\mathbf{w}^*(\boldsymbol{\theta}) = \underset{\mathbf{w}\in\mathbb{R}^p}{\arg\min}\, \mathcal{L}(\mathbf{w}, \boldsymbol{\theta}) \qquad \text{avec} \qquad \mathcal{L} = \frac{1}{k}\sum_{i=1}^{k}\left(L_i - \hat{L}_i\right)^2$$

Cette optimisation, non convexe en général, est réalisée par descente de gradient via l'algorithme Adam.

Le second niveau, bien plus complexe, consiste à identifier le jeu d'hyperparamètres $\boldsymbol{\theta}^*$ qui minimise la valeur de la fonction de perte atteinte après convergence. Notons $\boldsymbol{\theta} \in \Theta$ un vecteur d'hyperparamètres et définissons :

$$\begin{aligned} \mathcal{J} : \Theta &\longrightarrow \mathbb{R}^+ \\ \boldsymbol{\theta} &\longmapsto \mathcal{J}(\boldsymbol{\theta}) = \mathcal{L}\left(\mathbf{w}^*(\boldsymbol{\theta}), \boldsymbol{\theta}\right) \end{aligned}$$

Le problème revient alors à résoudre :

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}\in\Theta}{\arg\min}\, \mathcal{J}(\boldsymbol{\theta})$$

Or l'espace $\Theta$ est de dimension élevée et partiellement continu : une vingtaine d'hyperparamètres couvrant aussi bien des choix architecturaux (nature et nombre de couches, nombre de cellules, régularisation) que des paramètres d'entraînement (optimiseur, taux d'apprentissage, taille de batch, SNR, taille du jeu de données), dont certains prennent leurs valeurs dans $\mathbb{R}^+$, de sorte que $|\Theta| \to \infty$. La fonction $\mathcal{J}$ est de surcroît inconnue analytiquement : elle ne peut être évaluée qu'en un point $\boldsymbol{\theta}_i$, au prix d'un cycle complet d'entraînement et de validation. Avec $N$ le nombre de cycles réalisables dans le temps imparti, le problème se réduit à :

$$\boldsymbol{\theta}^* \approx \underset{\boldsymbol{\theta}\in\{\boldsymbol{\theta}_1,\ldots,\boldsymbol{\theta}_N\}}{\arg\min}\, \mathcal{J}(\boldsymbol{\theta}) \qquad \text{avec} \quad N \ll |\Theta|$$

Le nombre de configurations testables étant ainsi sévèrement contraint par le temps disponible face à l'infinité des possibilités, cette exploration a constitué la principale difficulté de la mission. Pour y répondre, les entraînements ont été lancés en continu, y compris la nuit et le week-end, afin de maximiser $N$. Mais au-delà du volume d'expériences, l'essentiel résidait dans la qualité de chaque essai : chaque configuration testée devait être le fruit d'une analyse réfléchie plutôt que d'un choix arbitraire. Avant chaque expérience, une hypothèse claire était formulée, fondée sur les résultats précédents et sur la compréhension du problème. Une configuration prometteuse qui réussit confirme l'hypothèse ; une configuration qui échoue doit permettre d'en comprendre les raisons et d'orienter l'exploration suivante. Dans ce processus, la part d'intuition était non négligeable : face à un espace de recherche aussi vaste et à une fonction $\mathcal{J}$ opaque, la capacité à identifier les directions les plus prometteuses relevait autant de l'expérience accumulée au fil des expérimentations que du raisonnement théorique.

*Reproductibilité des résultats de la littérature.*

Une partie significative du travail a consisté à implémenter des méthodes proposées dans différents articles. Si la plupart des publications détaillent l'architecture et les principaux hyperparamètres, certains éléments essentiels restent non spécifiés : détails de la méthode d'entraînement, initialisation des poids, ou formulation exacte de certaines équations. Ces informations manquantes se sont révélées déterminantes, et les performances annoncées n'ont pas pu être reproduites. Après concertation avec mon maître d'apprentissage, nous avons choisi d'abandonner ces pistes pour concentrer les efforts sur notre propre méthode. Ce choix s'est avéré payant : DeepMAP atteint des performances en BER équivalentes à celles de la littérature, avec une architecture plus légère et donc un temps de décodage plus court.

### Gestion des risques

Plusieurs risques ont été identifiés en amont et suivis tout au long de la mission. Le tableau 4 en présente une synthèse, accompagnée des mesures mises en place pour en limiter l'impact.

TABLE 4 – Gestion des risques de la mission

| Risque | Probabilité | Impact | Solution |
|---|---|---|---|
| Performances BER insuffisantes | Élevée | Élevé | Méthode propre après échec de la littérature |
| Temps d'inférence trop élevé | Faible | Élevé | Architecture légère (GRU + Conv1D) |
| GPU non exploitable | Avérée | Moyen | Entraînements sur CPU |
| Surapprentissage | Moyenne | Élevé | Validation hors distribution, suivi des courbes de perte |
| Temps de mission limité | Faible | Moyen | Entraînements en continu, priorisation par analyse |

## Résultats

### Validité des résultats

*Performance de décodage de DeepTurbo*

Une fois DeepMAP validé en tant que décodeur SISO, deux modules ont été assemblés en parallèle selon le schéma itératif du turbo décodage classique, formant ainsi DeepTurbo. La structure est fidèle à l'architecture du turbo décodeur présentée en figure 3 : chaque décodeur SISO est remplacé par un DeepMAP, tandis que l'entrelaceur, le désentrelaceur et le mécanisme d'échange d'information restent inchangés. Le réseau reçoit en entrée la trame bruitée, constituée de 1784 triplets, chaque triplet représentant les 3 bits encodés, le turbo-encodeur ayant un rendement $R = 1/3$. À chaque itération, les deux DeepMAP communiquent selon le schéma turbo, et l'information a posteriori $q$ gagne en précision. Après la dernière itération, l'information a posteriori finale est désentrelacée et une décision sur son signe fournit les bits décodés $\hat{u}_k$.

La figure 7 compare les courbes de BER de DeepTurbo et du turbo décodeur classique sur canal AWGN. Les deux courbes se superposent sur l'ensemble de la plage de SNR testée : DeepTurbo reproduit fidèlement les performances du turbo décodeur de référence. Ce résultat est remarquable car il démontre que l'approximation introduite par DeepMAP au niveau de chaque décodeur SISO ne se dégrade pas au fil des itérations. Au contraire, le mécanisme itératif d'échange d'information fonctionne aussi efficacement avec les réseaux de neurones qu'avec l'algorithme BCJR.

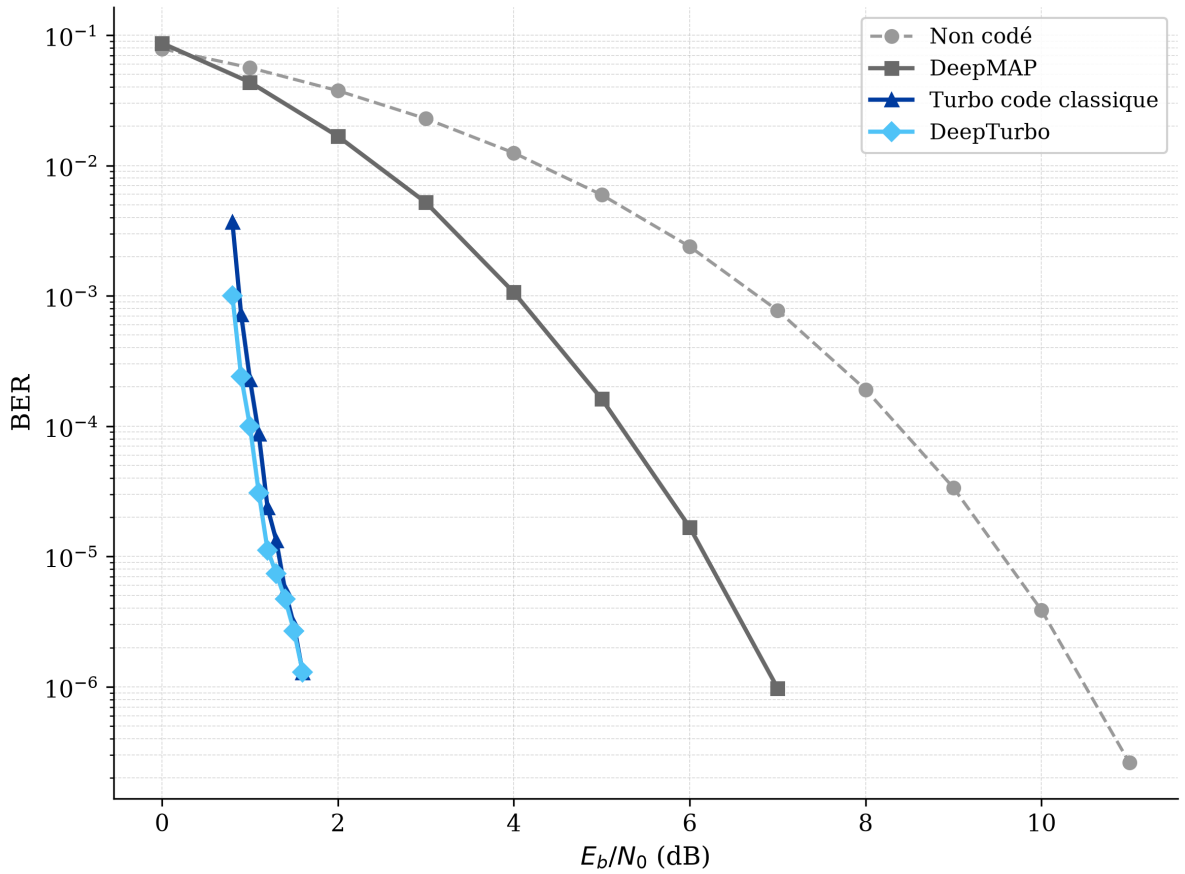



FIGURE 7 – Courbes de BER de DeepTurbo et du turbo décodeur classique sur canal AWGN

La figure 8 propose un zoom sur les courbes de BER de DeepTurbo et du turbo décodeur classique. On observe un léger écart à bas SNR, où DeepTurbo se révèle légèrement plus performant. Ce phénomène s'explique par la nature de l'approximation réalisée par DeepMAP. Avec une MAE de 0.34 par rapport au BCJR, les LLR prédits par le réseau diffèrent en moyenne du décodeur classique d'environ $\pm 0.34$. À bas SNR, le signal est très bruité et les LLR calculés par le turbo décodeur classique sont proches de zéro : le décodeur hésite sur la valeur de nombreux bits. Dans ces conditions, la perturbation de $\pm 0.34$ introduite par DeepMAP suffit à faire basculer certaines décisions du bon côté, ce qui se traduit par un BER ponctuellement meilleur.

À mesure que le SNR augmente, le bruit diminue et le turbo décodeur classique devient plus confiant dans ses décisions : les LLR atteignent des valeurs élevées en valeur absolue (de l'ordre de 60). La perturbation de $\pm 0.34$ devient alors négligeable face à ces amplitudes, et les deux courbes convergent vers les mêmes performances. Ce comportement confirme que l'approximation de DeepMAP est suffisamment fidèle pour ne pas dégrader le turbo décodage, et que le léger écart observé à bas SNR est un effet de bord bénéfique plutôt qu'un défaut.

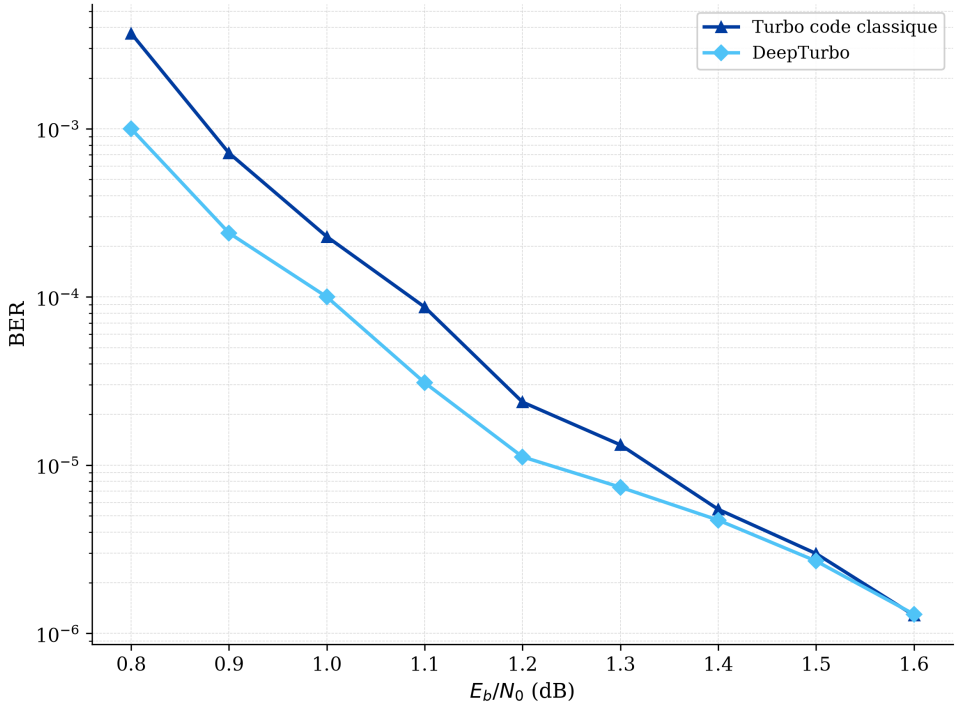



FIGURE 8 – Zoom sur les courbes de BER de DeepTurbo et du turbo décodeur classique

*Vitesse de décodage de DeepTurbo*

En termes de vitesse, le gain observé au niveau du décodeur SISO se répercute directement sur le turbo décodeur complet. La figure 9 compare les temps de décodage d'une trame de 1784 bits : le turbo décodeur classique nécessite 0.90 secondes contre 0.14 secondes pour DeepTurbo, soit un facteur d'accélération de 6.4. Ce gain s'explique par le fait que chaque itération, auparavant réalisée par l'algorithme BCJR, est désormais effectuée par un DeepMAP dont la prédiction repose sur de simples produits matriciels. Il convient de rappeler que ces mesures correspondent au décodage d'une seule trame. Dans un scénario opérationnel où plusieurs trames doivent être décodées simultanément, DeepTurbo pourrait les regrouper en un batch et les traiter en parallèle, réduisant davantage le temps de décodage par trame. Ces résultats valident pleinement l'approche : un réseau de neurones peut se substituer au turbo décodeur sans dégradation mesurable du BER, tout en offrant un gain significatif en vitesse de décodage.

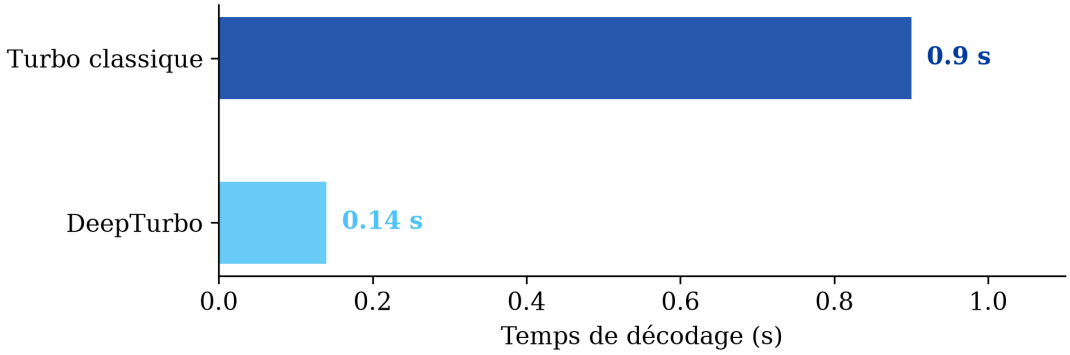



FIGURE 9 – Temps de décodage d'une trame de 1784 bits : DeepTurbo vs turbo décodeur classique

Il convient de nuancer ces résultats de vitesse. Les mesures ont été réalisées sur une même machine (MacBook Pro M2 Pro, CPU uniquement) et dépendent de la manière dont le turbo décodeur classique a été implémenté : le degré de vectorisation du code, les optimisations algorithmiques et

le langage utilisé influencent directement les temps de décodage. Pour obtenir une comparaison définitive, il faudrait implémenter les deux approches sur une même plateforme cible (FPGA ou déploiement C++ optimisé) et les confronter dans des conditions identiques. Cela dépasse le cadre de cette mission, mais les résultats obtenus fournissent une première indication encourageante du gain en vitesse apporté par l'approche par réseaux de neurones.

### Caractère innovant

L'approche développée se distingue des méthodes existantes sur plusieurs points. Contrairement aux trois écoles présentées dans l'état de l'art, qui reposent sur une initialisation par le BCJR, une spécialisation par itération ou un entraînement bout en bout, DeepMAP s'appuie exclusivement sur une phase d'optimisation pure des poids du réseau pour reproduire le décodeur SISO, sans aucune étape de fine-tuning. Cette simplicité de la méthode d'entraînement constitue un atout : elle réduit la complexité du pipeline, évite les instabilités liées au fine-tuning et produit néanmoins des performances en BER équivalentes à celles du turbo décodeur classique, avec une architecture plus légère et donc plus rapide.

Par ailleurs, cette mission représente la toute première exploration de méthodes de deep learning appliquées au turbo décodage au sein de Safran Data Systems. Jusqu'ici, les travaux de l'équipe CRT en matière de décodage reposaient exclusivement sur des approches algorithmiques classiques. En démontrant qu'un réseau de neurones peut reproduire fidèlement le turbo décodeur tout en offrant un gain en vitesse, ce travail fournit à l'équipe TDS une base de code, de résultats et de retours d'expérience sur laquelle de futurs travaux pourront s'appuyer. Il contribue également à positionner le deep learning comme une piste concrète dans la stratégie de transition du modem vers une offre logicielle.

### Utilité et réutilisabilité des résultats

Le gain en vitesse obtenu par DeepTurbo rend désormais envisageable le déploiement du turbo décodeur en logiciel, ce qui constituait le verrou identifié au début de cette mission. Concrètement, l'architecture de DeepMAP peut être exportée et réimplémentée en C++ par l'équipe de développement du CRT, les opérations du réseau (produits matriciels, convolutions 1D) étant supportées par des bibliothèques C++ standard.

De plus, les résultats présentés dans ce rapport ont été obtenus sur CPU. En environnement de production, l'exécution sur GPU accélérerait encore les produits matriciels et permettrait de traiter des batchs plus importants, ce qui laisse entrevoir un gain supplémentaire par rapport aux mesures actuelles.

### Limites et suites à donner

Les résultats obtenus s'inscrivent dans un cadre bien défini. Les performances de DeepTurbo ont été validées exclusivement sur canal AWGN, avec un turbo code de polynômes générateurs $(7, 5)$ et un entrelaceur fixé pour des trames de 1784 bits. La capacité du réseau à généraliser à d'autres conditions (canaux à évanouissement, polynômes générateurs différents, autres types d'entrelaceur) reste à explorer.

Par ailleurs, le déploiement en C++ n'a pas été réalisé dans le cadre de cette mission. Il constitue la prochaine étape naturelle, à mener par l'équipe de développement du CRT, afin de mesurer les performances en vitesse dans un environnement de production et de confirmer la viabilité de l'approche pour le modem logiciel.

Enfin, l'étude pourrait être étendue à d'autres configurations de turbo codes afin de vérifier si la méthode DeepMAP se transpose sans modification majeure, ou si chaque configuration nécessite un réentraînement spécifique du réseau.

### Quantification de la valeur ajoutée

L'objectif principal de la mission est atteint : un réseau de neurones reproduit les performances du turbo décodeur classique en termes de BER tout en offrant un gain significatif en vitesse de décodage. Ce résultat rend concrètement envisageable le déploiement du turbo décodeur en logiciel, avec les avantages que cela implique pour Safran Data Systems : réduction des coûts de production, suppression du matériel dédié, simplification du déploiement chez le client et raccourcissement des cycles de développement et de test.

Au-delà du résultat technique, cette mission a démontré que le décodage par réseau de neurones est une approche viable dans un contexte industriel. Ce constat ouvre la voie à de futures missions qui pourront s'appuyer sur le code développé, les résultats documentés et les retours d'expérience accumulés. Plusieurs prolongements sont envisageables : appliquer la même approche à d'autres codes correcteurs utilisés par l'équipe, explorer des architectures encore plus légères pour accroître le gain en vitesse, ou encore étudier le comportement du réseau sur d'autres modèles de canal. Cette mission constitue ainsi le point de départ d'un axe de recherche nouveau pour l'équipe TDS.

## Prolongements possibles

### TurboNet : un réseau unique non itératif

Dans l'architecture actuelle, DeepTurbo conserve la structure itérative du turbo décodeur classique : deux DeepMAP s'échangent des estimations de fiabilité sur plusieurs itérations. C'est précisément cette itérativité qui limite le gain en vitesse, car chaque itération doit s'achever avant que la suivante ne commence. Un prolongement naturel consisterait à remplacer l'ensemble de la chaîne itérative par un unique réseau de neurones, qui prendrait en entrée la trame bruitée et produirait directement les LLR a posteriori finaux en une seule passe.

Une telle architecture pourrait reposer entièrement sur des couches convolutives 1D empilées. Le réseau appliquerait une succession de convolutions :

$$\mathbf{h}^{(l)} = \varphi\left(\mathbf{W}^{(l)} * \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \qquad l = 1, \dots, L$$

où $\mathbf{h}^{(0)} = \mathbf{w} \in \mathbb{R}^{k\times 3}$ est la trame bruitée, $\mathbf{W}^{(l)}$ les filtres de la couche $l$ et $L$ le nombre total de couches. En augmentant progressivement la taille des noyaux ou le nombre de couches, le champ réceptif du réseau couvrirait l'ensemble de la trame, capturant les dépendances à longue portée aujourd'hui assurées par les itérations. La prédiction se réduirait alors à une unique séquence de produits matriciels, totalement parallélisable, ce qui pourrait réduire encore considérablement le temps de décodage.

### Turbo Auto-Encodeur

Les travaux présentés dans ce rapport supposent un turbo encodeur fixe et cherchent à en optimiser le décodeur. Une approche plus ambitieuse consisterait à remplacer également l'encodeur par un réseau de neurones, formant ainsi un auto-encodeur de bout en bout [12]. Dans ce cadre, le réseau encodeur apprendrait son propre code correcteur, conjointement optimisé avec le décodeur pour un canal donné. Cette approche, connue sous le nom de Turbo Auto-Encodeur, pourrait potentiellement découvrir des schémas de codage mieux adaptés aux contraintes spécifiques du système que les codes classiques conçus à la main.

### Généralisation à d'autres modèles de canal

L'ensemble des résultats de cette mission a été obtenu sur canal AWGN, où le bruit suit une loi gaussienne $\mathcal{N}(0, \sigma^2)$. Or, dans des conditions réelles de transmission, le bruit peut présenter des caractéristiques différentes : queues lourdes (distribution de Student), comportement asymétrique, ou perturbations non stationnaires difficiles à modéliser analytiquement. Un réseau

de neurones, de par sa capacité d'approximation universelle, pourrait être entraîné sur des données issues de ces canaux plus complexes et apprendre à décoder sans hypothèse explicite sur la distribution du bruit [8]. Explorer cette voie permettrait de vérifier si l'approche DeepMAP se généralise au-delà du cadre gaussien et offre une robustesse accrue face à des conditions de transmission réalistes.

## Bilan des apprentissages professionnels

### Compétences techniques acquises

En arrivant chez Safran Data Systems, mes connaissances en traitement du signal étaient limitées. Les codes correcteurs d'erreurs, le turbo codage, l'algorithme BCJR, les rapports de vraisemblance logarithmiques : autant de notions que j'ai dû m'approprier rapidement pour être opérationnel sur la mission. Cette montée en compétences s'est faite par un travail personnel soutenu sur la théorie, accompagné par Amine Jhaidri qui a su m'expliquer les subtilités du domaine avec pédagogie. En trois ans, je suis passé d'une connaissance superficielle du traitement du signal à une compréhension en profondeur des mécanismes du turbo codage, suffisante pour être capable d'en reproduire le comportement par un réseau de neurones.

Le deep learning constituait pour moi un sujet de passion que cette mission m'a permis d'approfondir considérablement, tant sur le plan théorique que pratique. Sur le plan mathématique, j'ai consolidé mes connaissances en optimisation non convexe, en descente de gradient stochastique, en rétropropagation, en régularisation et en théorie de l'approximation universelle. Sur le plan informatique, j'ai acquis une maîtrise pratique de l'écosystème NumPy, TensorFlow et Matplotlib. Cette mission m'a confronté à une grande variété d'architectures de réseaux de neurones : réseaux récurrents (RNN, Recurrent Neural Network ; GRU ; LSTM, Long Short-Term Memory) pour la capture des dépendances temporelles, réseaux convolutifs (CNN 1D, Convolutional Neural Network) pour l'extraction de motifs locaux, couches denses pour la régression, et réseaux bidirectionnels pour la propagation de l'information dans les deux sens de la séquence. J'ai appris à comprendre les forces et les limites de chacune de ces architectures, et à les combiner pour répondre à un problème d'apprentissage supervisé concret. Plus largement, cette mission m'a appris à faire le lien entre la théorie et l'ingénierie : partir d'un problème industriel, le formuler comme un problème de régression ou de classification, concevoir le jeu de données adapté, choisir les critères d'optimisation pertinents, et itérer jusqu'à obtenir un résultat exploitable.

Enfin, cette mission m'a formé aux pratiques de la recherche appliquée : lecture et analyse critique d'articles scientifiques sur Arxiv, implémentation de méthodes publiées, rédaction de rapports techniques et présentation de résultats devant des interlocuteurs aux profils variés.

### Compétences transversales

Au-delà des compétences techniques, cette alternance m'a permis de m'intégrer dans un environnement professionnel structuré. J'ai appris les codes de l'entreprise : rigueur dans la documentation, respect des processus internes, communication adaptée selon l'interlocuteur. Les points réguliers avec mon maître d'apprentissage m'ont appris à présenter des résultats de manière synthétique, à argumenter mes choix techniques et à accueillir les retours critiques pour réorienter mon travail lorsque nécessaire.

Le fait de travailler au sein d'une équipe de six personnes, tout en étant le seul sur un sujet de deep learning, m'a également appris à concilier autonomie et collaboration : mener mes recherches de façon indépendante tout en sollicitant les bonnes personnes au bon moment, qu'il s'agisse d'Amine Jhaidri pour le cadrage technique ou d'autres collaborateurs du groupe ayant travaillé sur des problématiques proches.

### Mise en perspective pour le futur professionnel

Travailler trois ans sur une même mission m'a appris la valeur du travail de fond. Sur un sujet aussi technique, c'est la compréhension profonde de chaque brique qui permet de les relier entre elles et d'acquérir une maîtrise globale du problème. J'ai pris l'habitude de ne jamais considérer un concept comme acquis sans l'avoir compris en détail, et c'est cette rigueur qui m'a permis, par exemple, d'identifier pourquoi certaines approches de la littérature ne fonctionnaient pas dans notre contexte, et de concevoir une méthode propre qui répondait aux contraintes de la mission.

Cette expérience m'a également appris la persévérance. Sur trois ans, un projet de recherche connaît naturellement des phases d'avancées rapides et des phases de plateau où les résultats progressent plus lentement. J'ai appris à rester méthodique dans ces moments, à chercher à comprendre l'origine d'un résultat inattendu plutôt qu'à l'écarter, et à garder la curiosité nécessaire pour explorer de nouvelles pistes. Cette capacité à tenir un effort de recherche sur la durée et à rester rigoureux quel que soit le rythme des avancées est sans doute l'apprentissage le plus précieux que je retire de cette mission.

## Prise de recul sur les enjeux éthiques

### Enjeux individuels et responsabilité de l'ingénieur

Travailler chez Safran Data Systems, une entreprise positionnée à l'intersection de la défense et du spatial, m'a amené à réfléchir à la responsabilité qui accompagne le métier d'ingénieur. Les modems conçus par l'équipe CRT servent à la transmission de données satellite dans des contextes civils comme militaires. En tant qu'ingénieur débutant, j'ai pris conscience que les choix techniques que l'on fait, même à l'échelle d'un algorithme de décodage, s'inscrivent dans une chaîne plus large dont les usages finaux dépassent le cadre de la mission. Cette prise de conscience implique de ne pas se limiter à la performance technique, mais de s'interroger sur le contexte dans lequel les résultats seront déployés et sur les conséquences qu'ils peuvent avoir.

### Enjeux collectifs et sociétaux

Le développement de systèmes de communication pour le secteur spatial et de la défense soulève des questions légitimes. Ces technologies contribuent à la souveraineté nationale et à la sécurité des communications, mais elles s'inscrivent aussi dans un écosystème industriel de défense dont les implications géopolitiques sont significatives. L'introduction du deep learning dans ce domaine ajoute une couche de complexité : un décodeur fondé sur un réseau de neurones est par nature moins interprétable qu'un algorithme classique comme le BCJR, dont le comportement est entièrement déterministe et démontrable mathématiquement. Cette question de l'interprétabilité et de la confiance dans les systèmes appris est un enjeu collectif important, en particulier dans des domaines où la fiabilité des communications est critique.

### Développement durable et RSE

L'entraînement de réseaux de neurones a un coût énergétique non négligeable. Dans le cadre de cette mission, des centaines d'entraînements ont été lancés, parfois en continu la nuit et le week-end, pour explorer l'espace des hyperparamètres. Chaque cycle consomme des ressources de calcul et donc de l'énergie. Ce constat m'a sensibilisé à l'importance de rationaliser les expériences : privilégier des architectures légères, réduire le nombre d'entraînements inutiles par une analyse préalable rigoureuse, et éviter les approches de type recherche exhaustive au profit d'une exploration guidée par la compréhension du problème. Il est à noter que le travail a été réalisé sur CPU (MacBook Pro M2 Pro), une configuration nettement moins énergivore que les clusters de GPU habituellement utilisés en deep learning, ce qui a limité l'empreinte environnementale des expérimentations.

### Diversité et interculturalité

L'équipe TDS, comme Safran Data Systems plus largement, rassemble des profils variés en termes de parcours académiques, d'origines et d'expériences professionnelles. Cette diversité se retrouve également à CentraleSupélec, où la formation m'a permis de côtoyer des étudiants issus d'horizons très différents. Dans le cadre de la mission, cette diversité s'est traduite concrètement par la richesse des échanges : les discussions avec des collègues ayant des expertises complémentaires (traitement du signal, FPGA, développement logiciel) ont nourri ma réflexion et élargi ma vision du problème au-delà du seul prisme du deep learning.

## Conclusion

Cette mission avait pour objectif d'explorer l'apport du deep learning pour accélérer le turbo décodage, dans le contexte de la transition du modem satellite de Safran Data Systems du FPGA vers une offre logicielle. Le travail a abouti à DeepMAP, un réseau de neurones capable de reproduire fidèlement le décodeur SISO, et à DeepTurbo, qui assemble deux DeepMAP en structure itérative pour reconstituer le turbo décodeur complet.

Les résultats obtenus répondent à la problématique posée. Sur canal AWGN, DeepTurbo égale les performances en BER du turbo décodeur classique, avec un gain significatif en vitesse de décodage. La faisabilité de l'approche, qui constituait la question centrale au démarrage de la mission, est désormais établie.

Ce travail ouvre plusieurs perspectives : le déploiement en C++ par l'équipe de développement, la conception d'un réseau unique non itératif pour accélérer encore le décodage, ou l'extension à d'autres modèles de canal et d'autres codes correcteurs. Plus largement, cette mission a démontré que le deep learning constitue une voie crédible pour la transition logicielle du modem, et a posé les premières bases d'un axe de recherche nouveau pour l'équipe TDS.

## Références bibliographiques


[1] C. Berrou, A. Glavieux et P. Thitimajshima, « Near Shannon limit error-correcting coding and decoding : Turbo-codes », in *Proceedings of the IEEE International Conference on Communications (ICC)*, t. 2, 1993, p. 1064-1070.

[2] E. Nachmani, Y. Be'ery et D. Burshtein, « Learning to decode linear codes using deep learning », *Allerton Conference*, 2016.

[3] T. Gruber, S. Cammerer, J. Hoydis et S. ten Brink, « On Deep Learning-Based Channel Decoding », *CISS*, 2017.

[4] H. Kim, Y. Jiang, R. Rana, S. Kannan, S. Oh et P. Viswanath, « Communication Algorithms via Deep Learning », *International Conference on Learning Representations (ICLR)*, 2018.

[5] S. A. Abrantes, « From BCJR to turbo decoding : MAP algorithms made easier », Technical report, 2004.

[6] L. Bahl, J. Cocke, F. Jelinek et J. Raviv, « Optimal decoding of linear codes for minimizing symbol error rate », *IEEE Transactions on Information Theory*, t. 20, n° 2, p. 284-287, 1974.

[7] L. Zhang, W. Fu, F. Shi et al., « A Parallel Turbo Decoder Based on Recurrent Neural Network », *Wireless Personal Communications*, t. 126, p. 975-993, 2022.

[8] Y. Jiang, H. Kim, H. Asnani, S. Kannan, S. Oh et P. Viswanath, « DeepTurbo : Deep Turbo Decoder », *arXiv preprint arXiv :1903.02295*, 2019.

[9] K. Cho et al., « Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation », *arXiv preprint arXiv :1406.1078*, 2014.

[10] F. Chollet, *L'apprentissage profond avec Python*. Paris, France : Eyrolles, 2018, isbn : 978-2491674007.

[11] I. Goodfellow, Y. Bengio et A. Courville, *Deep Learning*. Cambridge, MA : MIT Press, 2016, isbn : 978-0262035613.

[12] Y. Jiang, H. Kim, H. Asnani, S. Kannan, S. Oh et P. Viswanath, « Turbo Autoencoder : Deep learning based channel codes for point-to-point communication channels », *arXiv preprint arXiv :1911.03038*, 2019.

## Rappels sur les turbo codes

### Encodeur convolutif RSC (7, 5)

L'encodeur utilisé dans le turbo code de Safran Data Systems est un encodeur convolutif récursif systématique (RSC) défini par les polynômes générateurs $(7, 5)$ en notation octale, soit :

$$g_1(D) = 1 + D + D^2 \qquad g_2(D) = 1 + D^2$$

où $D$ représente l'opérateur de retard. Sa fonction de transfert s'écrit :

$$G(D) = \left[1,\ \frac{g_2(D)}{g_1(D)}\right]$$

Le premier terme correspond à la sortie systématique ($x_1 = u_k$), le second à la sortie de parité. Le caractère récursif provient de la division par $g_1(D)$, qui introduit une boucle de rétroaction dans les registres à décalage. L'encodeur possède une mémoire $m = 2$, soit $2^m = 4$ états dans le treillis.

### Structure de l'entrelaceur

L'entrelaceur $\Pi$ est une permutation déterministe de taille $k = 1784$ bits, qui réordonne les bits d'entrée de sorte que deux bits voisins pour le premier encodeur se retrouvent éloignés pour le second. Le désentrelacement s'effectue par la permutation inverse $\Pi^{-1}$. En décorrélant les motifs d'erreurs entre les deux encodeurs, l'entrelaceur garantit que les informations échangées lors du décodage itératif sont statistiquement indépendantes, condition nécessaire à la convergence du turbo décodeur.

### Algorithme BCJR

L'algorithme BCJR (Bahl, Cocke, Jelinek et Raviv) calcule, pour chaque bit $u_k$, la probabilité a posteriori conditionnellement au signal reçu $\mathbf{y}$. Il opère sur le treillis de l'encodeur RSC en trois passes.

À chaque transition $s' \to s$ du treillis est associée une métrique de branche $\gamma_k(s', s)$, qui quantifie la vraisemblance de cette transition. La passe forward parcourt le treillis de gauche à droite et calcule récursivement la probabilité $\alpha_k(s)$, résumant l'information des observations passées :

$$\alpha_k(s) = \sum_{s'} \alpha_{k-1}(s') \cdot \gamma_k(s', s)$$

La passe backward le parcourt en sens inverse et calcule symétriquement $\beta_k(s')$, résumant l'information des observations futures :

$$\beta_k(s') = \sum_{s} \beta_{k+1}(s) \cdot \gamma_{k+1}(s', s)$$

Le LLR a posteriori est ensuite obtenu en combinant ces trois quantités :

$$L(u_k|\mathbf{y}) = \log \frac{\displaystyle\sum_{(s',s)|u_k=1} \alpha_{k-1}(s') \cdot \gamma_k(s', s) \cdot \beta_k(s)}{\displaystyle\sum_{(s',s)|u_k=0} \alpha_{k-1}(s') \cdot \gamma_k(s', s) \cdot \beta_k(s)}$$

où les sommes portent sur les transitions associées à $u_k = 1$ et $u_k = 0$.

### Information extrinsèque

Le LLR a posteriori se décompose en trois termes distincts :

$$\underbrace{L(u_k|\mathbf{y})}_{\text{a posteriori}} = \underbrace{L(u_k)}_{\text{a priori}} + \underbrace{L_c y_1}_{\text{canal}} + \underbrace{L_e(u_k)}_{\text{extrinsèque}}$$

Le premier terme est l'information a priori fournie par l'autre décodeur. Le deuxième est la contribution directe du canal sur le bit systématique reçu, pondérée par la fiabilité du canal $L_c$. Le troisième, l'information extrinsèque, représente ce que le décodeur a appris de nouveau sur $u_k$ en exploitant uniquement les bits de parité, indépendamment de ce qu'il savait déjà.

C'est cette information extrinsèque qui est échangée entre les deux décodeurs. En ne transmettant que l'apport propre de chaque décodeur, on évite qu'un décodeur ne se voie renvoyer sa propre information, ce qui nuirait à la convergence. Elle est obtenue par soustraction :

$$L_e(u_k) = L(u_k|\mathbf{y}) - L(u_k) - L_c y_1$$

En reprenant les notations $p$ (a priori) et $q$ (a posteriori) du corps du rapport, l'a priori transmis à l'itération suivante s'écrit :

$$p_{t+1} = q_t - p_t - L_c y_1$$

## Protocole d'évaluation BER

Les courbes de BER présentées dans ce rapport sont obtenues par simulation Monte Carlo. Le principe est le suivant : pour chaque valeur de SNR, un grand nombre de trames sont générées aléatoirement, encodées par le turbo encodeur, bruitées selon le modèle AWGN, puis décodées. Le BER est ensuite calculé en comparant les bits décodés aux bits originaux sur l'ensemble des trames simulées.

La fiabilité de l'estimation dépend directement du nombre de trames simulées : plus ce nombre est élevé, plus l'estimation du BER est précise. En pratique, la simulation se poursuit pour chaque point de SNR jusqu'à collecter un nombre suffisant d'erreurs (au moins une centaine), ce qui garantit une estimation statistiquement fiable. Ce critère est particulièrement important à haut SNR, où le BER est très faible et où il faut simuler davantage de trames pour observer des erreurs.

## Glossaire

| | |
|---|---|
| **AWGN** | *Additive White Gaussian Noise* |
| **BCJR** | Algorithme de Bahl, Cocke, Jelinek et Raviv |
| **BER** | *Bit Error Rate* |
| **BPSK** | *Binary Phase Shift Keying* |
| **CNN** | *Convolutional Neural Network* |
| **FPGA** | *Field-Programmable Gate Array* |
| **GRU** | *Gated Recurrent Unit* |
| **LLR** | *Log-Likelihood Ratio* |
| **LSTM** | *Long Short-Term Memory* |
| **MAE** | *Mean Absolute Error* |
| **MAP** | *Maximum A Posteriori* |
| **MSE** | *Mean Squared Error* |
| **RNN** | *Recurrent Neural Network* |
| **RSC** | Codeur Récursif Systématique Convolutif |
| **SISO** | *Soft-In Soft-Output* |
| **SNR** | *Signal-to-Noise Ratio* |